More practice - Streaming, scale and concurrency

1 questions to study here, each on its own page, with the full interview answer and code

Nothing to skip: every question that fits this topic is covered here

1
MEDIUM

How to use

- Questions run from Easy to Medium to Hard, newest last-asked date first inside each level.
- PDFs run from the one with the most questions to the fewest. A question has pages in only one PDF; the others list it as SKIP
- Every page has < Prev | Contents | Next > at the bottom. Tap a title in the Contents to jump to it.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

More practice - Streaming, scale and concurrency | 1 questions | tap a title to jump to it

M 1. Design an elevator control system **PracHub 2025-09-06**

STUDY PATH: question 160 of 167 (more practice)

Priority score 1.0 | Streaming, scale and concurrency | asked 409 days ago

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 1 of 1

Design an elevator control system

POSTED 2025-09-06 | 409 DAYS AGO | OLDER

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 221/228

ABOUT THIS QUESTION

Topic: More practice - Streaming, scale and concurrency

Optimal answer from: written and tested here for this exact question

LeetCode match: No direct match (OOP / low-level design)

Posted on PracHub: 2025-09-06 (updated 2026-10-07)

Original: https://prachub.com/interview-questions/design-an-elevator-control-system-2

QUESTION

Design the software for a multi-elevator control system in a high-rise building. Specify classes, scheduling algorithms for car assignments, handling peak traffic, fairness, fault tolerance, and safety constraints. Consider real-time constraints and a simulation and testing strategy.

APPROACH

Model Elevator, Request and Controller classes; scheduling by nearest-car or SCAN/LOOK direction-based assignment; queue targets per elevator, handle peak traffic, fairness, fault tolerance and safety; discuss simulation and testing strategy.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how many elevators and floors, whether to design only the control logic or also the hardware, request types (hall call with direction, destination inside), and what to optimize (wait time, fairness, energy).

BRUTE FORCE TO SAY OUT LOUD

A single queue: serve requests first come first served with the nearest idle car. Easy but wasteful.

OPTIMAL IDEA

Classes: Request, Elevator (floor, direction, state, a set of stops), Controller (assigns requests). Use SCAN/LOOK inside each car: keep moving in the current direction serving stops, then reverse. Assign a hall call to the car with the lowest cost, such as distance plus a penalty if it is moving away or is full. Add a step method that advances time so it can be simulated and tested.

DRY-RUN THESE CASES

One request while idle, a request in the opposite direction, two cars equally close, a car out of service with pending stops, the top and bottom floors.

INTERVIEW-DAY PLAN

LIKELY FOLLOW-UPS

Peak traffic modes, weight limits, fault tolerance (reassign stops on failure), and safety (doors, overload).

PITFALLS

Putting all logic in one class. Not saying how you will test it.

HOW TO THINK ABOUT IT

Separate three things: the data (Request), one car's behaviour (Elevator: floor, direction, stops, door/state) and the policy that decides which car takes a call (Controller). Say the scheduling rule out loud: a car keeps going in its direction serving stops, then reverses (SCAN/LOOK), which avoids the back-and-forth of first-come-first-served. Add a step() so the whole thing can be simulated and tested.

Basis: Written object-oriented design (the classic low-level design question). A SCAN (elevator) scheduling policy per car with a cost-based assignment of hall calls.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
requests = deque()  # arrival order
def run():
    while requests:
        req = requests.popleft()
        car = nearest_idle_car(req.floor)
        car.go_to(req.floor)
        car.go_to(req.destination)
```

TIME	SPACE
O(requests * cars)	O(requests)

WHY IT IS SLOW

Cars zig-zag across the building and requests wait behind far-away ones, so total travel and waiting are high. Serving stops in sweep order (SCAN) and assigning by cost cuts both.

OPTIMAL SOLUTION · Elevator (SCAN) per car and cost-based assignment of calls

TIME	SPACE
O(cars) per call, O(1) per car per step	O(stops)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Peak traffic (morning lobby rush, evening exodus).

Add modes: during the morning rush park idle cars in the lobby and let them go up with a full load; in the evening do the opposite. Zone cars (low floors, high floors) or use destination dispatch (passengers enter their floor first) to group riders by destination.

FOLLOW-UP 2 Fairness and starvation.

SCAN already guarantees every stop is reached within one sweep. For hall calls add an aging term to the cost so a long-waiting call wins over a closer new one.

FOLLOW-UP 3 Safety and fault tolerance.

Keep door and overload as explicit states; a car that fails is marked out of service and its stops are reassigned (the controller does this), and the controller should be restartable from persistent state of cars and pending calls.

FOLLOW-UP 4 Morning rush: idle cars should wait at the lobby.

When a car has no stops, give it a stop at the lobby (or a configured parking floor per car). Peak modes are just policies layered on the same step() and cost-based assignment; the elevator algorithm does not change.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# MAIN QUESTION: Design an elevator control system
#
# Design the software for a multi-elevator control system
# in a high-rise building. Specify classes, scheduling
# algorithms for car assignments, handling peak traffic,
# fairness, fault tolerance, and safety constraints.
# Consider real-time constraints and a simulation and
# testing strategy.
#
# FOLLOW-UPS (answered below, in this order):
# 1. Peak traffic (morning lobby rush, evening exodus).
# 2. Fairness and starvation.
# 3. Safety and fault tolerance.
# 4. Morning rush: idle cars should wait at the lobby.
#
# INTERVIEW NOTES
# Separate three things: the data (Request), one car's
# behaviour (Elevator: floor, direction, stops,
# door/state) and the policy that decides which car takes
# a call (Controller). Say the scheduling rule out loud: a
# car keeps going in its direction serving stops, then
# reverses (SCAN/LOOK), which avoids the back-and-forth of
# first-come-first-served. Add a step() so the whole thing
# can be simulated and tested.
#
# BRUTE FORCE (say it first): First-come-first-served on
# the nearest idle car

# Serve requests in arrival order, each by whichever car
# is closest at that moment, visiting stops in the order
# they were requested. Easy to explain and fair, but cars
# zig-zag across the building. Time O(requests * cars).
# Space O(requests).


# BRUTE FORCE (say it first): First-come-first-served on
# the nearest idle car | time O(requests * cars)
def fcfs_distance(stops, start=0):
    pos, total = start, 0
    for s in stops:
        total += abs(s - pos)
        pos = s
    return total


# HELPERS for the base question
```

```
INF = float("inf")


class Request:
    def __init__(self, floor, direction):
        # direction: +1 up, -1 down
        self.floor, self.direction = floor, direction


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id, self.floor = car_id, floor
        # 0 = idle, +1 = going up, -1 = going down
        self.direction = 0
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        # how bad is it to give this car the request?
        if not self.in_service:
            return INF
        dist = abs(self.floor - req.floor)
        if self.direction == 0:
            return dist
        # on the way: the car is moving toward the floor
        # and
        # the call goes the same way
        gap = req.floor - self.floor
        ahead = gap * self.direction >= 0
        if ahead and req.direction == self.direction:
            return dist
        # otherwise it finishes its sweep first
        span = max(self.stops) - min(self.stops) + 1
        return dist + 2 * span + 10

    def step(self):
        # one tick: serve a stop here, then move one floor
        served = None
        if self.floor in self.stops:
```

```
            self.stops.discard(self.floor)
            served = self.floor
        if not self.stops:
            self.direction = 0
            return served
        up = [s for s in self.stops if s > self.floor]
        down = [s for s in self.stops if s < self.floor]
        # keep the direction while there are stops ahead
```

```
        # (SCAN)
        if (self.direction >= 0 and up) or (
                self.direction < 0 and not down):
            self.direction = 1
        else:
            self.direction = -1
        self.floor += self.direction
        return served


# BASE QUESTION (optimal): Elevator (SCAN) per car and
# cost-based assignment of calls
# Idea: Each car keeps a set of stops and a direction.
# step() moves one floor in its direction and removes a
# stop it reaches; when no stops remain ahead it reverses,
# and when none remain at all it goes idle.
# FOLLOW-UP 1: Peak traffic (morning lobby rush, evening
# exodus).
# Answer: Add modes: during the morning rush park idle
# cars in the lobby and let them go up with a full load;
# in the evening do the opposite.
class Controller:
    def __init__(self, cars, lobby=0):
        self.cars, self.lobby = cars, lobby

    def assign(self, req):
        # the cheapest in-service car takes the call
        car = min(self.cars, key=lambda c: c.cost(req))
        if car.cost(req) == INF:
            return None
        car.stops.add(req.floor)
        return car.id

    def tick(self):
        return {c.id: c.step() for c in self.cars}

    # FOLLOW-UP 3: a failed car hands its stops over
```

```
    def fail(self, car_id):
        # a broken car: take it out and hand its stops to
        # others
        car = next(c for c in self.cars if c.id == car_id)
        car.in_service = False
        orphans, car.stops = car.stops, set()
        for floor in orphans:
            self.assign(Request(floor, 0))

    # FOLLOW-UP 1, 4: morning rush: idle cars wait in the
    # lobby
    def park_idle(self):
        # morning rush: idle cars wait in the lobby
        for car in self.cars:
            if car.in_service and not car.stops:
                car.stops.add(self.lobby)


if __name__ == "__main__":
    a, b = Elevator("A", 0), Elevator("B", 10)
    ctl = Controller([a, b])
    # the closest idle car takes the call
    assert ctl.assign(Request(2, 1)) == "A"
    assert ctl.assign(Request(9, -1)) == "B"
    for _ in range(3):
        served = ctl.tick()
    assert a.floor == 2 and served["A"] in (None, 2)
    assert 2 not in a.stops
    # a failed car hands its stops to the other car
    b.stops = {8}
    ctl.fail("B")
    assert not b.in_service and 8 in a.stops
    assert ctl.assign(Request(1, 1)) == "A"
    # nobody in service: the request cannot be taken
    a.in_service = False
```

```
    assert ctl.assign(Request(1, 1)) is None
    # idle cars go back to the lobby
    c = Elevator("C", 5)
    Controller([c]).park_idle()
    assert c.stops == {0}
```